

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**



BÁO CÁO ĐỒ ÁN

THIẾT KẾ HỆ THỐNG MÃ HÓA DES SOC

Giảng viên hướng dẫn:

TRƯỜNG VĂN CƯỜNG

Sinh viên thực hiện:

VÕ THÁI DUY - 23520398

NGUYỄN CHÍ THANH – 23521448

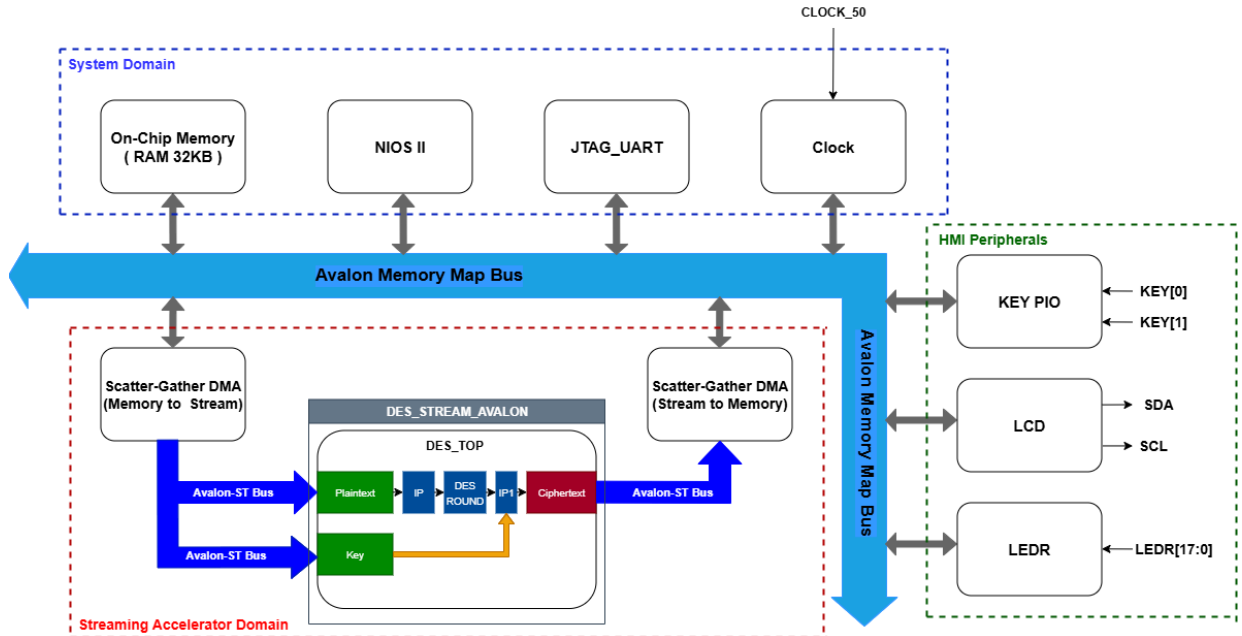
Github: https://github.com/DuyVo-Hcm/DES64_Soc_V2

Mục lục

I. Tổng quan hệ thống Soc	3
1. Sơ đồ khối hệ thống SoC.....	3
2. Giao diện Qsys (Platform Designer)	4
II. Chi tiết hệ thống Soc	6
1. Phần cứng (Hardware)	6
1.1. Lỗi mã hóa DES_core	6
1.2. Khối bọc DES_STREAM_AVALON	9
1.3. Scatter-Gather DMA	11
1.4. RAM tĩnh (On-Chip Memory - 32KB)	12
1.5. Ngoại vi Hiển thị LCD 16x2	13
1.6. Ngoại vi Nút nhấn KEY (Input PIO)	13
1.7. Ngoại vi LEDR (Output PIO)	13
2. Phần mềm (Firmwawe)	14
2.1. Cấu trúc hàm DES_Encrypt_With_SGDMA.....	14
2.2. Hàm hoán vị swap_bytes_32()	15
2.3. Hàm kiểm thử tự động Run_All_10_Testcases_Automatically()	15
2.4. Khối điều khiển màn hình LCD	17
2.5. Hàm main ().....	18
III. Hiện thực hệ thống trên DE2 KIT	20
IV.Đánh giá	21
1. Phân công công việc	21
2. Hướng phát triển trong tương lai	21
V. TÀI LIỆU THAM KHẢO.....	22

I. Tổng quan hệ thống Soc

1. Sơ đồ khối hệ thống SoC



Hệ thống mã hóa theo thuật toán DES trên SoC được phân làm 3 vùng:

System Domain

- Phân vùng này đóng vai trò làm nhân điều phối toàn bộ tài nguyên, quản lý các máy trạng thái logic (State Machines) của phần mềm và thiết lập cấu hình khởi tạo cho toàn bộ hệ thống SoC.

Bên trong vùng này bao gồm:

- Vi xử lý NIOS II: Đóng vai trò là bộ vi xử lý trung tâm (CPU). Nó thực thi mã C, quản lý các trạng thái hoạt động (State Machine), quét nút nhấn và cấu hình các gói lệnh Descriptor cho SGDMA.
- On-Chip Memory (RAM 32KB): Bộ nhớ tĩnh tích hợp trên chip. Đây là nơi lưu trữ mã máy chương trình (.text), mảng 10 bộ Testcases, vùng đệm tạm (Stack) và đặc biệt là nơi chứa các Descriptor của SGDMA.
- JTAG_UART: Ngoại vi giám sát nâng cao. Nó đóng vai trò làm cầu nối truyền thông điệp trực tiếp từ bộ nhớ RAM lên Terminal của máy tính qua cáp nạp, giúp hiển thị log đối soát kết quả.
- Khởi Clock: Nhận xung nhịp cứng CLOCK_50 (50MHz) từ chân chip FPGA, phân phối và đồng bộ tần số làm việc cho CPU, Bus hệ thống và lõi mã hóa DES để toàn mạch chạy đồng pha.

Streaming Accelerator Domain

- Phân vùng chuyên dụng đảm nhận nhiệm vụ xử lý dữ liệu nặng (Mã hóa thuật toán DES) với băng thông tối đa mà không tiêu tốn chu kỳ xử lý của CPU. Bên trong vùng này bao gồm:

- Scatter-Gather DMA Tx & Rx: Cặp động cơ vận chuyển dữ liệu tự động. Tx làm nhiệm vụ lấy Plaintext và Key từ RAM đưa vào lõi DES; Rx làm nhiệm vụ nhận Ciphertext từ

lỗi DES đưa về RAM.

- Khối DES_STREAM_AVALON (Lỗi DES_TOP): Khối sở hữu các tầng Pipeline thực thi hoán vị IP, 16 vòng Feistel băm dữ liệu với Khóa con (Key Schedule), và hoán vị ngược IP⁻¹. Giao tiếp hoàn toàn bằng Bus luồng Avalon-ST.

HMI Peripherals Domain

- Phân vùng này đóng vai trò là cầu nối vật lý (Physical Layer Interfacing), thực hiện nhiệm vụ thu thập tín hiệu điều khiển từ môi trường thực tế và hiển thị trực quan các trạng thái logic của hệ thống. Bên trong vùng này bao gồm:

- KEY PIO: Đọc trạng thái Active-Low của các phím cơ học. Cụ thể là dùng KEY[1] để đổi Testcase và KEY[0] để ra lệnh bắt đầu mã hóa hoặc reset.
- LCD: Nhận lệnh điều khiển qua thuật toán Bit-banging I2C (chân SCL/SDA) để xuất chuỗi Hex kết quả hiển thị trực quan.
- LEDR : Gồm 18 bóng LED, sẽ nhận lệnh ghi trực tiếp từ CPU để sáng đồng loạt (0x3FFFF) làm tín hiệu trực quan báo hiệu "Đã hoàn thành chu kỳ SGDMA".

2. Giao diện Qsys (Platform Designer)

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
✓		clock	Clock Source	clk reset	clock				
✓		nios2_qsys_0	Nios II Processor	clk reset_n data_master instruction_master jtag_debug_module_re... jtag_debug_module custom_instruction_m...	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clock [clk] [clk] [clk] [clk] [clk] [clk]	IRQ 0	IRQ 31	
✓		onchip_memory2_0	On-Chip Memory (RAM or ROM)	clk1 s1 reset1	Double-click to export Double-click to export Double-click to export	clock [clk1] [clk1]	# 0x0000_8000	0x0000_ffff	
✓		jtag_uart_0	JTAG UART	clk reset avalon_jtag_slave	Double-click to export Double-click to export Double-click to export	clock [clk] [clk]	# 0x0001_1180	0x0001_118f	
✓		DES_STREAM_AVALO...	DES_STREAM_AVALON	clock_sink avalon_streaming_sink avalon_streaming_sou...	Double-click to export Double-click to export Double-click to export	clock [clock_sink] [clock_sink] [clock_sink]			
✓		sgdma_tx	Scatter-Gather DMA Controller	clk reset csr descriptor_read descriptor_write m_read out	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clock [clk] [clk] [clk] [clk] [clk] [clk]	# 0x0001_1080	0x0001_10bf	
✓		sgdma_rx	Scatter-Gather DMA Controller	clk reset csr descriptor_read descriptor_write in m_write	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clock [clk] [clk] [clk] [clk] [clk] [clk]	# 0x0001_10c0	0x0001_10ff	
✓		SDA	PIO (Parallel I/O)	clk reset s1 external_connection	Double-click to export Double-click to export Double-click to export	clock [clk] [clk]	# 0x0001_1170	0x0001_117f	
✓		SCL	PIO (Parallel I/O)	clk reset s1 external_connection	Double-click to export Double-click to export Double-click to export	clock [clk] [clk]	# 0x0001_1160	0x0001_116f	
✓		KEY	PIO (Parallel I/O)	clk reset s1 external_connection	Double-click to export Double-click to export Double-click to export	clock [clk] [clk]	# 0x0001_1150	0x0001_115f	
✓		LEDR	PIO (Parallel I/O)	clk reset s1 external connection	Double-click to export Double-click to export Double-click to export	clock [clk] [clk]	# 0x0001_1140	0x0001_114f	

Bảng ánh xạ địa chỉ

Tên ngoại vi	Địa chỉ cơ sở	Kích thước	Chức năng
onchip_memory2_0	0x0000_8000 -> 0x0000_FFFF	32KB	Lưu trữ Firmware (Code C), Stack, mảng 10 Testcases và các Descriptor của SGDMA.
sgdma_tx (csr)	0x0001_1080 -> 0x0001_10BF	64 Bytes	Thanh ghi cấu hình điều khiển luồng truyền DMA từ RAM sang ngoại vi.
sgdma_rx (csr)	0x0001_10C0 -> 0x0001_10FF	64 Bytes	Thanh ghi cấu hình điều khiển luồng nhận DMA từ ngoại vi về RAM.
SDA (PIO)	0x0001_1170 -> 0x0001_117F	16 Bytes	Chân dữ liệu (Serial Data) phục vụ thuật toán Bit-banging giao tiếp I2C điều khiển LCD.
SCL (PIO)	0x0001_1160 -> 0x0001_116F	16 Bytes	Chân xung nhịp (Serial Clock) điều khiển đồng bộ màn hình LCD I2C.
KEY (PIO)	0x0001_1150 -> 0x0001_115F	16 Bytes	Cổng vào đọc trạng thái nút nhấn cơ học (KEY[1] đổi bộ dữ liệu, KEY[0] chốt chạy/Reset).
LEDR (PIO)	0x0001_1140 -> 0x0001_114F	16 Bytes	Cổng xuất tín hiệu kích sáng thanh 18 bóng LED đỏ báo hiệu hoàn tất phiên mã hóa thành công.
jtag_uart_0	0x0001_1188 -> 0x0001_118F	8 Bytes	Kênh truyền thông nạp và xuất dữ liệu đối soát lỗi (Log Terminal) lên màn hình máy tính.
DES_STREAM_AVALON		0 Bytes	Lỗi tăng tốc DES (Streaming IP): Kết nối thuần túy bằng bus Avalon-ST, không chiếm không gian bộ nhớ Avalon-MM.

- Đây là cấu hình chi tiết hệ thống SoC trên Qsys. Hệ thống có trục bus Avalon-MM phân phối địa chỉ rõ ràng cho RAM 32KB tại vùng 0x8000 và các ngoại vi HMI PIO tại vùng 0x11140 đến 0x11170. Riêng lõi cứng DES_STREAM_AVALON không tiêu tốn không gian địa chỉ hệ thống, nó được cô lập hoàn toàn trong phân vùng Streaming và

chỉ kết nối trực tiếp với cặp khối sgdma_tx/sgdma_rx bằng Bus luồng Avalon-ST để đạt tốc độ xử lý tối đa.

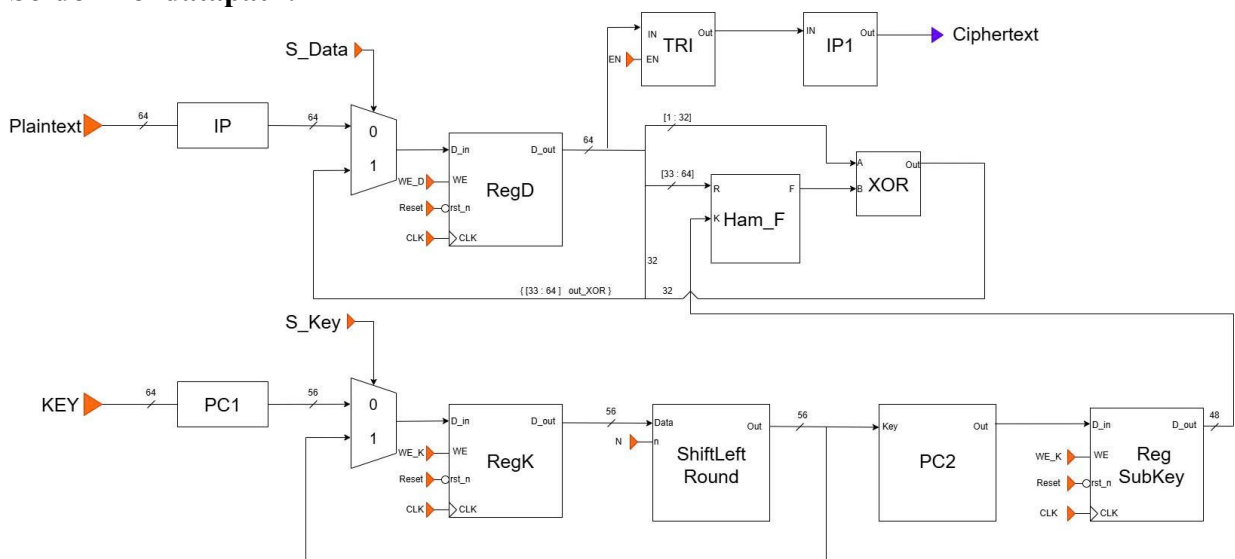
- Các GIO dù không sử dụng hết bộ nhớ nhưng vẫn cấp 16 Bytes nhớ vì khối PIO Core của Altera luôn được thiết kế cố định gồm 4 thanh ghi chức năng độc lập là: Data, Direction, Interrupt Mask, và Edge Capture để CPU cấu hình. Vì vi xử lý Nios II và trực bus Avalon-MM hoạt động theo kiến trúc 32-bit, nên mỗi thanh ghi này bắt buộc phải chiếm một không gian aligned chuẩn 4 Bytes bộ nhớ. Phần cứng chỉ thực sự sử dụng bit cần thiết của các thanh ghi này cho chân ngoại vi, các bit còn lại được giữ làm dự phòng.

II. Chi tiếp hệ thống Soc

1. Phần cứng (Hardware)

1.1. Lỗi mã hóa DES_core

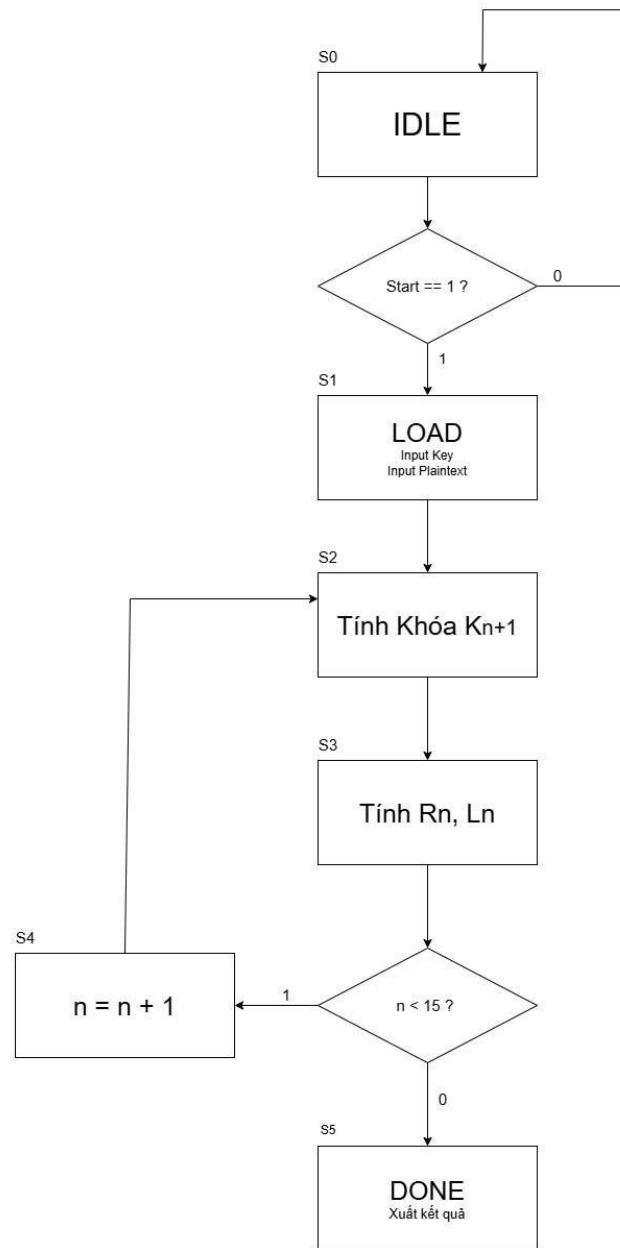
Sơ đồ khối datapath:



 Các khối cần thiết kế:

- Khối IP: Hoán vị khởi tạo
- Khối RegD (Reg 64 bit): Thanh ghi lưu trữ Left, Right từng vòng
- Khối biến đổi dữ liệu hàm F: Hàm mã hóa chính
- Khối PC1: Hoán vị Key
- Khối RegK (Reg 56 bit): Thanh ghi lưu trữ Key
- Khối ShiftLeft Round: Dịch trái vòng
- Khối PC2: Hoán vị Key
- Khối RegSubKey (Reg 48 bit): Lưu giá trị của K_{n+1}
- Khối Tri: Cho phép xuất dữ liệu khi mã hóa xong
- Khối IP1: Hoán vị khởi tạo đảo

Máy trạng thái:

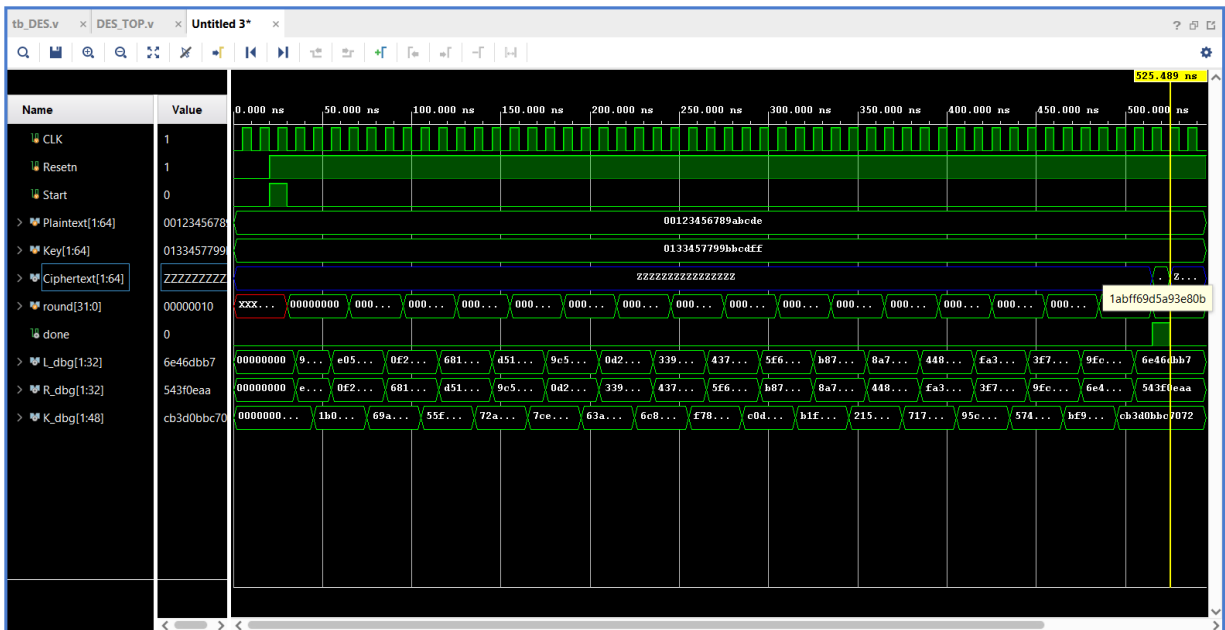


Ý nghĩa các state:

- S0: Khởi tạo giá trị ban đầu, đợi start để bắt đầu
- S1: Nhập giá trị Plaintext và Key vào thanh ghi lưu trữ
- S2,S3,S4: Thực hiện tính khóa vòng, tính Left, Right từng round, FSM sẽ có một biến đếm n, khởi tạo $n = 0$, biến đếm này đảm bảo đủ 16 vòng lặp.
- S5: Cho phép xuất giá trị Ciphertext

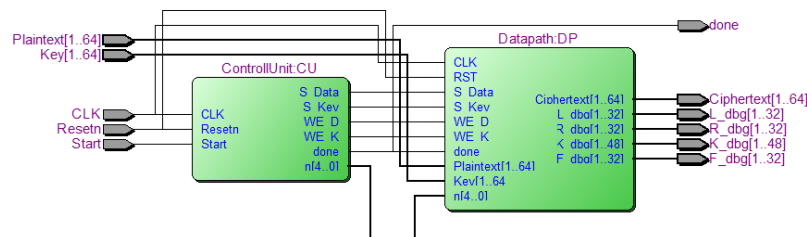
Số chu kỳ dự kiến để có kết quả: $S0 + S1 + 16(S2, S3, S4) + S5 = 51$ (chu kỳ)

Kết quả waveform:



- Từ 0ns đến 20ns: Do Resetn = 1 và Start = 0, nên mạch chưa hoạt động.
- Từ 30ns đến 530: Mạch hoạt động, mỗi 3 chu kỳ sẽ có giá trị Left, Right, Subkey mới, giống như dự đoán FSM, sau 51 chu kỳ sẽ có kết quả.

RTL Viewer:



Tần số Fmax:

DES_TOP.v		Compilation Report - doan												
Table of Contents		Slow Model Fmax Summary												
- Flow Summary - Flow Settings - Flow Non-Default Global Settings		<table> <thead> <tr> <th></th><th>Fmax</th><th>Restricted Fmax</th><th>Clock Name</th><th>Note</th></tr> </thead> <tbody> <tr> <td>1</td><td>227.17 MHz</td><td>227.17 MHz</td><td>CLK</td><td></td></tr> </tbody> </table>				Fmax	Restricted Fmax	Clock Name	Note	1	227.17 MHz	227.17 MHz	CLK	
	Fmax	Restricted Fmax	Clock Name	Note										
1	227.17 MHz	227.17 MHz	CLK											

- Fmax = 227.17 MHz: Đây là Tần số hoạt động tối đa (Maximum Frequency) của lõi DES_TOP. Điều này có nghĩa là thiết kế phần cứng của thuật toán DES này có thể chạy trơn tru với nhịp clock lên tới khoảng 227 triệu chu kỳ mỗi giây.

1.2. Khối bọc DES_STREAM_AVALON



- Sơ đồ trên minh họa kiến trúc đóng gói của module DES_STREAM_AVALON. Lõi tính toán DES_CORE (màu đỏ) được đặt làm trung tâm, ẩn hoàn toàn khỏi hệ thống bus bên ngoài. Lớp vỏ bọc (màu xám) đóng vai trò là cầu nối chuyển đổi từ luồng dữ liệu nối tiếp 32-bit (chuẩn Avalon-ST) sang dữ liệu song song 64-bit để cấp cho lõi DES.

Bảng tóm tắt tín hiệu:

Nhóm Hệ thống (System)

- CLK (In): Xung nhịp đồng bộ hệ thống.
- Reset_n (In): Tín hiệu Reset toàn mạch (tích cực mức thấp 0).

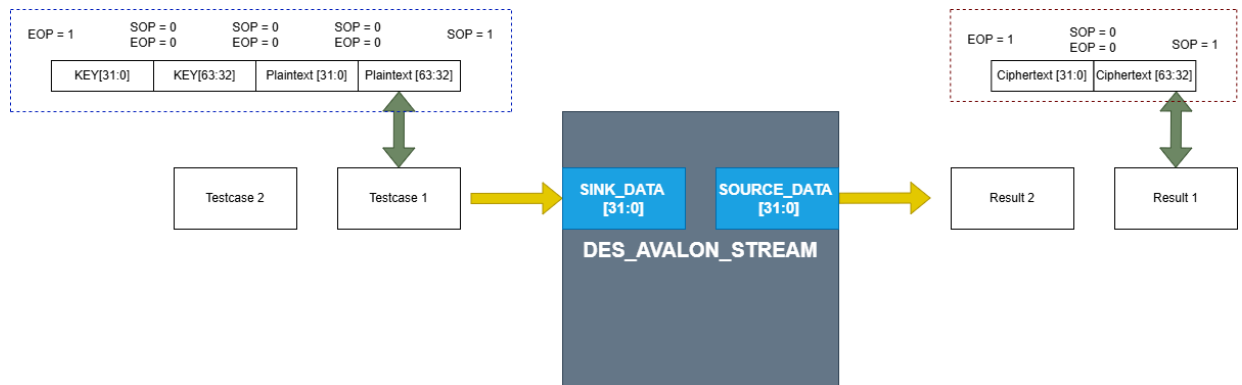
Nhóm Ngõ vào - SINK (Nhận dữ liệu từ DMA Tx)

- sink_data [31:0] (In): Bus nhận dữ liệu (Plaintext và Key).
- sink_valid (In): Cờ xác nhận dữ liệu hợp lệ do hệ thống truyền (DMA) cấp. Khi bằng 1, báo hiệu dữ liệu đang có mặt trên bus Sink_data là dữ liệu thật, không phải rác.
- sink_startofpacket (In): Cờ SOP - Đánh dấu Word đầu tiên của gói.
- sink_endofpacket (In): Cờ EOP - Đánh dấu Word cuối cùng của gói.
- sink_ready (Out): DES báo lại DMA đang trống, có thể đưa dữ liệu vào. (Kéo xuống 0 để ép DMA dừng truyền khi đang bận tính toán).

Nhóm Ngõ ra - SOURCE (Phát dữ liệu về DMA Rx)

- source_data [31:0] (Out): Bus xuất kết quả mã hóa (Ciphertext).
- source_valid (Out): Cờ xác nhận dữ liệu hợp lệ do module DES tự tạo ra. Khi bằng 1, báo cho DMA biết kết quả tính toán đã hoàn tất.
- source_startofpacket (Out): Cờ SOP - Đánh dấu nửa 32-bit cao của kết quả.
- source_endofpacket (Out): Cờ EOP - Đánh dấu nửa 32-bit thấp, chốt gói.
- source_ready (In): DMA Rx báo sẵn sàng nhận dữ liệu từ DES. (Nếu DMA chưa sẵn sàng, DES phải giữ kết quả lại chờ).

Mô tả luồng dữ liệu (Datastream In/Out)



Quá trình Datastream IN

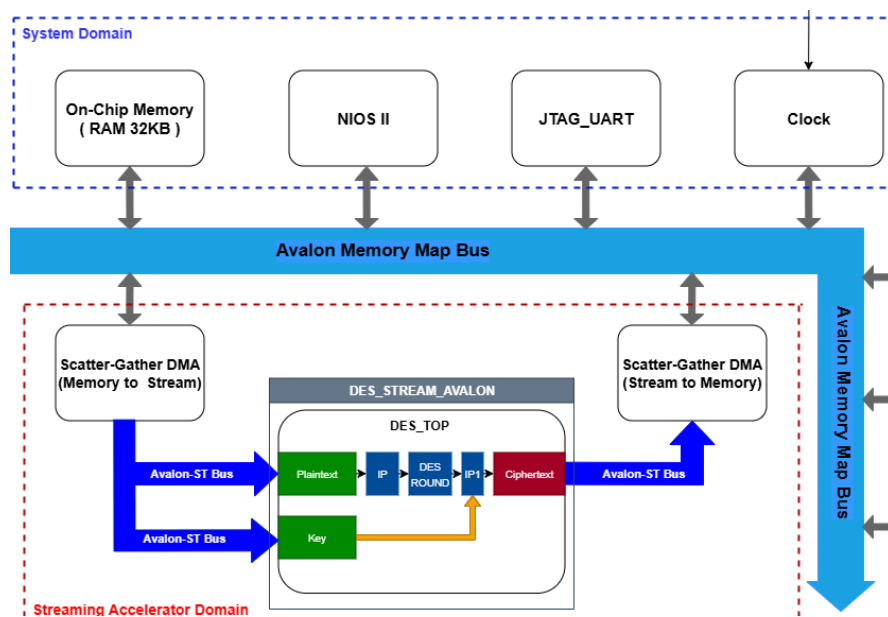
- Dữ liệu đầu vào DES_AVALON_STREAM cần phải có Plaintext [63:0] và Key [63:0] nhưng đường ngõ vào Sink_Data [31:0] chỉ có 32-bit. Quá trình ghép 4 mảnh 32-bit để thành 128-bit được quản lý bởi 2 tín hiệu SOP (sink_startofpacket) và EOP (sink_endofpacket).
 - Clock 1 (Start of Packet): Mảnh 32-bit cao của bản rõ (Plaintext [63:32]) được nạp vào đầu tiên, đi kèm cờ nhận diện ranh giới SOP = 1. Mạch FSM bên trong sẽ dựa vào cờ này để bắt đầu chu kỳ thu thập dữ liệu mới.
 - Clock 2 & Clock 3 (Payload): Lần lượt nạp mảnh 32-bit thấp của bản rõ (Plaintext [31:0]) và mảnh 32-bit cao của khóa (Key [63:32]). Lúc này các cờ SOP và EOP đều ở mức 0.
 - Clock 4 (End of Packet): Mảnh 32-bit thấp của khóa (Key [31:0]) đi vào cuối cùng, đi kèm cờ EOP = 1. Tín hiệu này chốt hạ gói tin, khóa cổng nạp và lập tức kích hoạt lõi DES bên trong tiến hành mã hóa.

Quá trình Datastream OUT

- Dữ liệu đầu ra tương tự, cần đẩy kết quả mã hóa Ciphertext[63:0] lên RAM. Được chia thành 2 mảnh 32-bit ghép thành 64-bit được quản lý bởi 2 tín hiệu SOP (source_startofpacket) và EOP (source_endofpacket).
 - Clock 1 (Start of Packet): Nửa 32-bit cao của bản mã (Ciphertext [63:32]) được xuất ra trước, đánh dấu bằng cờ SOP = 1.
 - Clock 2 (End of Packet): Nửa 32-bit thấp của bản mã (Ciphertext [31:0]) được xuất ra ngay sau đó, đánh dấu bằng cờ EOP = 1 để kết thúc quá trình truyền. Dữ liệu này lập tức được DMA Rx đưa về RAM và lưu lại thành Result 1.

- sink_ready là tín hiệu báo cho SGDMA biết có được phép gửi dữ liệu tiếp hay không. Mạch sẽ khóa cửa (sink_ready = 0) nếu rơi vào 1 trong 4 trường hợp busy sau đây:
 - packet_ready: Đã gom đủ 4 mảnh 32-bit, chuẩn bị mã hóa.
 - start_reg: Đang kích xung nhịp Start cho lõi DES.
 - des_running: Lõi DES đang tính toán mã hóa bên trong.
 - output_pending: Đã tính xong nhưng DMA Rx đầu ra chưa nhận hết kết quả.

1.3. Scatter-Gather DMA



Lý do dùng Scatter-Gather DMA thay vì DMA thường:

- DMA tiêu chuẩn (Standard DMA) bắt buộc vi xử lý phải can thiệp cấu hình lại từ đầu sau mỗi lần truyền xong một khối dữ liệu.
- SGDMA hoạt động dựa trên cơ chế Chuỗi mệnh lệnh (Descriptors). Vi xử lý NIOS II chỉ cần tạo ra một danh sách các công việc lưu vào RAM và cấp quyền điều khiển cho SGDMA. Động cơ này sẽ tự động đọc chuỗi lệnh và thực thi việc vận chuyển dữ liệu ở tốc độ tối đa của phần cứng.

Khối SGDMA Tx (Memory to Stream - Đầu vào)

- Nằm ở phía bên trái sơ đồ, khối DMA này chịu trách nhiệm cấp nguồn dữ liệu cho lõi mã hóa.
 - Đầu vào (Mũi tên xám trở xuống): Giao tiếp với bộ nhớ RAM qua chuẩn Avalon Memory-Mapped. Nó tự động đọc mảng dữ liệu Testcase (gồm Plaintext và Key) do CPU NIOS II chuẩn bị sẵn.
 - Đầu ra (Các mũi tên xanh lam đậm): Đóng vai trò là cầu nối giao thức, SGDMA này chuyển đổi dữ liệu tĩnh thành luồng dữ liệu động (Streaming) qua bus Avalon-ST. Sơ đồ thể hiện rõ hai nhánh mũi tên xanh lam bơm trực tiếp vào hai khối thành ghi Plaintext và Key bên trong lớp vỏ bọc DES_STREAM_AVALON. Việc truyền tải này diễn ra hoàn toàn tự động dựa trên chuỗi lệnh Descriptor mà không tốn chu kỳ xử lý của CPU.

Khối SGDMA Rx (Stream to Memory - Đầu ra)

- Nằm ở phía bên phải sơ đồ, khối DMA này làm nhiệm vụ thu gom kết quả trả về cho hệ thống.
 - Đầu vào (Mũi tên xanh lam đậm): Nhận luồng kết quả Ciphertext 64-bit vừa được tính toán xong từ cổng Avalon-ST của module DES.
 - Đầu ra (Mũi tên xám trở lên): Khối SGDMA thực hiện chuyển đổi giao thức ngược lại (từ Stream về Memory). Nó tự động đóng gói kết quả, tìm đến địa chỉ vùng nhớ đích (`dma_dest_buffer`) trên RAM và ghi đè dữ liệu vào đó qua chuẩn Avalon Memory-Mapped. Sau khi hoàn tất, nó sẽ lật cờ báo cáo trạng thái để CPU NIOS II có thể tiến hành đối soát kết quả hiển thị lên màn hình.

1.4. RAM tĩnh (On-Chip Memory - 32KB)

- Trong hệ thống này, On-Chip Memory là vùng nhớ RAM tĩnh (SRAM) được tổng hợp trực tiếp từ các khối M9K/M20K bên trong bản thân chip FPGA.
- Trong kiến trúc hệ thống nhúng này, bắt buộc phải dùng RAM thay vì ROM vì 3 lý do cốt lõi:
 - Lưu trữ dữ liệu biến động (Variables & Stack): ROM là bộ nhớ chỉ đọc. Trong khi đó, vi xử lý NIOS II cần một nơi để liên tục cập nhật các biến số, cờ trạng thái và không gian ngăn xếp (Stack) khi thực thi các hàm C.
 - Vùng đệm cho luồng DMA (Buffers): Cấp DMA yêu cầu không gian bộ nhớ hai chiều. CPU cần ghi dữ liệu vào mảng nguồn (`dma_src_buffer`), và kênh DMA Rx cần ghi đè kết quả bản mã trả về vào mảng đích (`dma_dest_buffer`). ROM không thể cho phép thao tác ghi theo thời gian thực này.
 - Tính linh hoạt khi nạp Code (Debugging): Việc lưu trữ toàn bộ chương trình trên RAM cho phép cấp JTAG nạp thẳng mã lệnh C từ máy tính xuống chip FPGA để chạy thử ngay lập tức. Nếu dùng ROM, mỗi lần sửa một dòng code bạn sẽ phải tốn thời gian tổng hợp và "đốt" (burn) lại phần cứng.

Việc chọn dung lượng 32KB là bài toán cân bằng tài nguyên (Resource Balancing) giữa không gian lưu trữ phần mềm và giới hạn phần cứng của chip FPGA:

- Nếu quá nhỏ (Dưới 32KB): Chỉ riêng các thư viện nền tảng (HAL) và thư viện C chuẩn (`stdio.h`, hàm `sprintf...`) đã chiếm khoảng 15-20KB. Nếu khai báo quá nhỏ, quá trình biên dịch sẽ báo lỗi tràn bộ nhớ (Memory Full) hoặc gây tràn ngăn xếp (Stack Overflow) khi chạy thực tế.
- Nếu quá lớn (Từ 64KB trở lên): RAM trên FPGA không phải RAM rời, mà được tạo ra từ các khối nhớ vật lý có số lượng cực kỳ giới hạn (như M9K Blocks) bên trong chip. Nếu chia quá nhiều RAM cho vi xử lý, chip sẽ cạn kiệt bộ nhớ để cấp cho các vùng đệm (FIFO) của kênh SGDMA, gây lỗi khi tổng hợp mạch (Compile).
- 32KB là mức tối ưu: Cung cấp vừa đủ không gian an toàn cho mã lệnh (~20KB), mảng hằng số 10 Testcases (~3KB) và vùng đệm DMA/Stack (~8KB), đồng thời vẫn chừa lại đủ khối M9K cứng cho các module phần cứng khác hoạt động.

1.5. Ngoại vi Hiển thị LCD 16x2

- Trong thiết kế này, màn hình LCD 16x2 (gắn kèm module I2C PCF8574) không sử dụng IP Core I2C phần cứng chuyên dụng. Thay vào đó, hệ thống áp dụng kỹ thuật **Bit-banging** thông qua hai khối PIO cơ bản là SCL và SDA nhằm tối ưu hóa tối đa tài nguyên logic (Logic Elements) của chip FPGA.
- Hai tín hiệu I2C cốt lõi là xung nhịp đồng bộ (**SCL**) và đường truyền dữ liệu (**SDA**) được nối trực tiếp vào hai khối PIO độc lập cấu hình ở ngõ ra (Output).
- Vi xử lý NIOS II xem hai chân này như hai ô nhớ tĩnh trên bus Avalon-MM (tại địa chỉ 0x11160 và 0x11170). Việc xuất mức điện áp 0V hay 3.3V ra chân vật lý của FPGA được thực hiện đơn giản thông qua lệnh ghi bộ nhớ (IOWR).

1.6. Ngoại vi Nút nhấn KEY (Input PIO)

- Khối KEY đóng vai trò là ngõ vào điều khiển vật lý (Physical Control Interface), cho phép người dùng tương tác trực tiếp để định tuyến hoạt động của hệ thống SoC. Cấu trúc và cơ chế hoạt động được chia thành các khía cạnh sau:
- Cấu hình PIO: Khối ngoại vi này sử dụng lõi Parallel I/O được cấu hình chuyên biệt làm ngõ vào (Input) tại địa chỉ cơ sở 0x11150 trên bus Avalon-MM.
- Logic Tích cực thấp (Active-Low): Thiết kế mạch phần cứng của các nút nhấn cơ học thường đi kèm điện trở kéo lên (Pull-up resistor). Do đó, tín hiệu hoạt động theo logic ngược: Khi trạng thái nghỉ (chưa nhấn), chân FPGA nhận mức logic 1; khi có tác động nhấn phím, mạch nối mass, chân FPGA nhận mức logic 0. CPU NIOS II sẽ đọc trạng thái này qua lệnh IORD_ALTERA_AVALON_PIO_DATA.
- Chức năng các phím:
 - KEY[0]: Resetrn
 - KEY[1]: Start

1.7. Ngoại vi LEDR (Output PIO)

- Cấu hình PIO: Hệ thống sử dụng một lõi Parallel I/O (PIO) tiêu chuẩn, được thiết lập ở chế độ Ngõ ra (Output-only). Độ rộng của cổng được cấu hình là 18-bit, tương ứng với số lượng 18 bóng LED vật lý có sẵn trên board mạch phát triển.
- Định tuyến giao tiếp: Khối PIO này được ánh xạ vào không gian nhớ Avalon-MM tại địa chỉ cơ sở 0x0001_1140. Phía đầu ra, cổng external_connection được gán trực tiếp vào các chân tín hiệu vật lý (Pin Assignment) điều khiển nguồn cấp cho dải đèn LED. Một bộ chốt (Latch) phần cứng bên trong PIO sẽ giữ nguyên mức điện áp xuất ra cho đến khi có chu kỳ ghi dữ liệu mới.

2. Phần mềm (Firmwave)

Phần mềm điều khiển hệ thống được phát triển bằng ngôn ngữ C trên môi trường Nios II EDS, dựa trên nền tảng thư viện trừu tượng phần cứng (HAL) của Altera. Kiến trúc phần mềm được chia thành các phân hệ xử lý chuyên biệt, từ điều khiển ngoại vi mức thấp đến quản lý luồng dữ liệu DMA mức cao.

2.1. Cấu trúc hàm DES_Encrypt_With_SGDMA

```
void DES_Encrypt_With_SGDMA(unsigned int pt_high, unsigned int pt_low, unsigned
int key_high, unsigned int key_low, unsigned int *ct_high, unsigned int *ct_low) {
    if (!sgdma_tx_dev || !sgdma_rx_dev) return;

    // 1. Gán dữ liệu thô
    dma_src_buffer[0] = swap_bytes_32(pt_high);
    dma_src_buffer[1] = swap_bytes_32(pt_low);
    dma_src_buffer[2] = swap_bytes_32(key_high);
    dma_src_buffer[3] = swap_bytes_32(key_low);

    dma_dest_buffer[0] = 0;
    dma_dest_buffer[1] = 0;

    // 2. Ép dữ liệu từ Cache CPU xuống RAM để DMA không đọc nhầm số 0
    alt_dcache_flush((void*)dma_src_buffer, sizeof(dma_src_buffer));
    alt_dcache_flush((void*)dma_dest_buffer, sizeof(dma_dest_buffer));

    // 3. Khởi tạo cấu trúc luồng
    alt_avalon_sgdma_construct_stream_to_mem_desc(&rx_descriptor, NULL, (alt_u32
*)dma_dest_buffer, 8, 0);
    alt_avalon_sgdma_construct_mem_to_stream_desc(&tx_descriptor, NULL, (alt_u32
*)dma_src_buffer, 16, 0, 1, 1, 0);

    // 4. Kích hoạt DMA
    alt_avalon_sgdma_do_async_transfer(sgdma_rx_dev, &rx_descriptor);
    alt_avalon_sgdma_do_async_transfer(sgdma_tx_dev, &tx_descriptor);

    // 5. Đọc kết quả qua con trỏ Uncached (Bypass Cache)
    volatile alt_u32 *uncached_dest = (volatile alt_u32 *)(((alt_u32)dma_dest_buffer) |
0x80000000);
    *ct_high = swap_bytes_32(uncached_dest[0]);
    *ct_low = swap_bytes_32(uncached_dest[1]);
}
```

Đây là hàm cốt lõi đóng vai trò điều khiển toàn bộ quá trình giao tiếp giữa bộ nhớ RAM và bộ tăng tốc DES phần cứng thông qua 2 kênh Scatter-Gather DMA. Quy trình hoạt động của hàm gồm các bước tuần tự:

- Gán dữ liệu: Nạp 128-bit dữ liệu (Plaintext và Key) vào mảng nguồn `dma_src_buffer`.
- Đồng bộ Cache (Xả bộ đệm): Gọi hàm `alt_dcacheflush()` để ép vi xử lý đẩy dữ liệu từ bộ đệm Cache cục bộ xuống bộ nhớ RAM vật lý. Điều này đảm bảo hệ thống DMA (vốn chỉ đọc từ RAM) không lấy nhầm dữ liệu rỗng.
- Cấu hình luồng (Descriptors): Thiết lập `tx_descriptor` (đọc 16-byte từ RAM đẩy vào DES) và `rx_descriptor` (nhận 8-byte kết quả từ DES ghi vào RAM).
- Truyền không đồng bộ: Kích hoạt 2 kênh DMA bằng lệnh `do_async_transfer`. Lúc này phần cứng tự động chạy mà không cần CPU can thiệp.
- Đọc kết quả: Để lấy bản mã (Ciphertext) về, CPU sử dụng con trỏ `uncached_dest` (bằng cách OR địa chỉ với `0x80000000`) nhằm bỏ qua Cache, ép CPU đọc trực tiếp kết quả mới nhất từ RAM.

2.2. Hàm hoán vị `swap_bytes_32()`

```
unsigned int swap_bytes_32(unsigned int val) {
    return ((val >> 24) & 0x000000FF) | ((val >> 8) & 0x0000FF00) |
           ((val << 8) & 0x00FF0000) | ((val << 24) & 0xFF000000);
}
```

- Hàm này được xây dựng để giải quyết bài toán lệch chuẩn lưu trữ (Endianness) giữa phần mềm và phần cứng:
- Nguyên lý: Vi xử lý Nios II hoạt động theo chuẩn Little-Endian (Byte thấp lưu trước), trong khi lõi DES phần cứng lại yêu cầu dữ liệu theo chuẩn Big-Endian (Byte cao đưa vào trước).
- Cách thực hiện: Hàm sử dụng các phép toán dịch bit (`>>`, `<<`) kết hợp với mặt nạ logic (`&`) để đảo ngược vị trí của 4 byte bên trong một biến 32-bit. Toàn bộ dữ liệu trước khi đẩy xuống DMA và sau khi đọc về từ RAM đều phải đi qua hàm này để đảm bảo tính chính xác của bản mã.

2.3. Hàm kiểm thử tự động `Run_All_10_Testcases_Automatically()`

```
typedef struct {
    unsigned int pt_high;
    unsigned int pt_low;
    unsigned int key_high;
    unsigned int key_low;
    unsigned int expected_ct_high;
    unsigned int expected_ct_low;
} DES_Testcase;

DES_Testcase testcases[10] = {
    {0x01234567, 0x89ABCDEF, 0x13345779, 0x9BBCDFF1, 0x85E81354, 0x0F0AB405}, // TC1
    {0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x8CA64DE9, 0xC1B123A7}, // TC2
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x7359B216, 0x3E4EDC58}, // TC3
    {0x30303030, 0x30303030, 0xAAA55AA5, 0xAAF2E12666, 0x675B082A}, // TC4
    {0xFEEBDAED, 0xDEADBEEF, 0x01010101, 0x01010101, 0xFC7FFA55, 0x6A4F527E}, // TC5
    {0x11111111, 0x11111111, 0xF0F0F0F0, 0xF0F0F0F0, 0xEE281E51, 0x5E9EAA27}, // TC6
    {0x12345678, 0x9ABCDEF0, 0x0F1E2D3C, 0x4B5A6978, 0x719FF2E2, 0x79207B4}, // TC7
    {0x55555555, 0x55555555, 0xCCCCCCCC, 0xCCCCCCCC, 0xF7B9CCD7, 0x8AC23A8F}, // TC8
    {0xAAAAA222, 0xB BBBB333, 0x11112222, 0x33334444, 0xE1C799F5, 0x146868AA}, // TC9
    {0x88888888, 0x77777777, 0x99999999, 0xB BBBB BBBB, 0x9ABB39FD, 0x518CEAB3} // TC10
};
```

```

void Run_All_10_Testcases_Automatically() {
    unsigned int ct_high, ct_low;
    int success_count = 0;
    char lcd_msg[20], buf_disp[17];

    for(int tc = 0; tc < 10; tc++) {
        // 1. Hiển thị trạng thái đang tính toán lên LCD
        LCD_Clear();
        sprintf(lcd_msg, "RUNNING TC: %02d", tc + 1);
        LCD_SetCursor(0, 0); LCD_String(lcd_msg);
        LCD_SetCursor(1, 0); LCD_String("CALCULATING...");

        // 2. Bơm dữ liệu vào phần cứng và chờ kết quả từ DMA
        DES_Encrypt_With_SGDMA(testcases[tc].pt_high, testcases[tc].pt_low,
                                testcases[tc].key_high, testcases[tc].key_low,
                                &ct_high, &ct_low);

        // 3. Chuẩn bị giao diện đối soát
        LCD_Clear();
        sprintf(lcd_msg, "TC%02d: ", tc + 1);
        LCD_SetCursor(0, 0); LCD_String(lcd_msg);

        // 4. Thuật toán kiểm tra (Verification)
        if(ct_high == testcases[tc].expected_ct_high &&
           ct_low == testcases[tc].expected_ct_low) {
            LCD_SetCursor(0, 6); LCD_String("SUCCESS");
            success_count++;
        } else {
            LCD_SetCursor(0, 6); LCD_String("FAILED ");
        }

        // 5. In chuỗi Hex kết quả thực tế lên dòng 2 của LCD
        hex_to_string_16char(ct_high, ct_low, buf_disp);
        LCD_SetCursor(1, 0); LCD_String(buf_disp);

        // Trễ 1.5s để người dùng kịp quan sát từng Testcase
        usleep(1500000);
    }

    // 6. Tổng kết điểm số sau khi hoàn thành chu trình
    LCD_Clear();
    LCD_SetCursor(0, 0); LCD_String("DONE ALL 10 TCs");
    sprintf(lcd_msg, "SCORE: %02d/10 OK ", success_count);
    LCD_SetCursor(1, 0); LCD_String(lcd_msg);
}

```

- Thay vì phải nhập tay từng trường hợp, phần mềm cung cấp một động cơ tự động chạy và đối soát kết quả để chứng minh độ tin cậy của phần cứng:
- Nguồn dữ liệu: Hệ thống khai báo sẵn một mảng cấu trúc (Struct Array) chứa 10 kịch bản kiểm thử, bao gồm đầu vào và bản mã kỳ vọng (Expected Ciphertext).
- Vòng lặp kiểm thử: Hàm sử dụng vòng lặp for chạy đủ 10 lần. Ở mỗi lần lặp, nó tự động trích xuất Testcase, gọi hàm DES_Encrypt_With_SGDMA để phần cứng giải quyết, sau đó so sánh kết quả trả về với kết quả chuẩn.
- Báo cáo: Kết quả đối soát (Đúng/Sai) của từng nhịp được in chi tiết ra màn hình Terminal (qua JTAG UART) và cập nhật tiến độ liên tục lên màn hình LCD. Cuối chương trình, hệ thống sẽ chốt điểm số tổng (ví dụ: SCORE: 10/10 OK).

2.4. Khởi điều khiển màn hình LCD

```
void I2C_SCL_Write(int s) {
    IOWR(SCL_BASE, 0, s);
}

void I2C_SDA_Write(int s) {
    if (s) IOWR(SDA_BASE, 1, 0);
    else { IOWR(SDA_BASE, 1, 1); IOWR(SDA_BASE, 0, 0); }
}

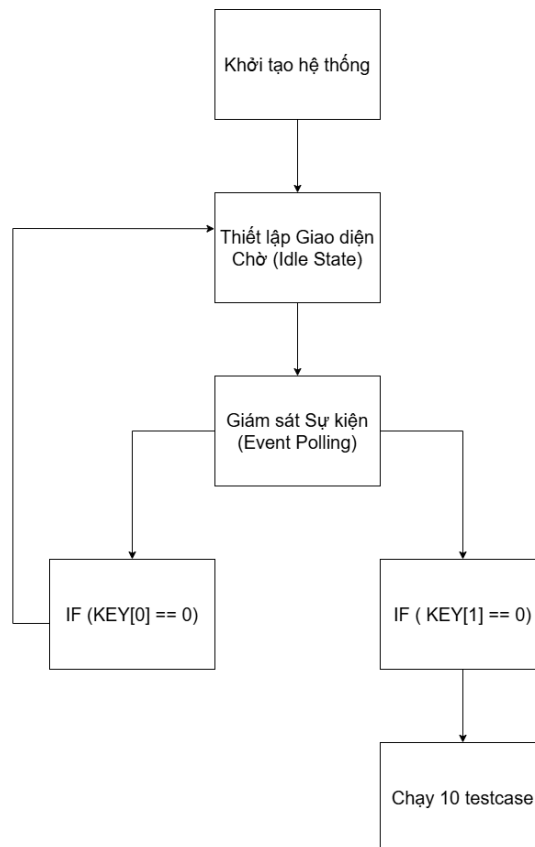
void I2C_Start() {
    I2C_SDA_Write(1); I2C_SCL_Write(1); delay_us(4);
    I2C_SDA_Write(0); delay_us(4); I2C_SCL_Write(0);
}

void I2C_SendByte(unsigned char d) {
    // Dịch lần lượt 8-bit dữ liệu từ MSB đến LSB
    for (int i = 0; i < 8; i++) {
        I2C_SDA_Write((d & 0x80) ? 1 : 0); delay_us(2);
        // Tạo sườn lên SCL để chốt bit
        I2C_SCL_Write(1); delay_us(4);
        I2C_SCL_Write(0); delay_us(2);
        d <<= 1;
    }
    I2C_SDA_Write(1); delay_us(2);
    I2C_SCL_Write(1); delay_us(4); I2C_SCL_Write(0);
}

void LCD_Write_Nibble(unsigned char n, unsigned char rs) {
    unsigned char d = n | (rs ? 0x01 : 0x00) | PIN_BL;
    I2C_Start();
    I2C_SendByte(I2C_LCD_ADDR << 1); // Gửi địa chỉ của chip PCF8574
    I2C_SendByte(d | PIN_EN);         // Kéo cờ EN (Enable) lên mức 1
    delay_us(1);
    I2C_SendByte(d & ~PIN_EN);        // Kéo cờ EN xuống mức 0 để chốt dữ liệu
    delay_us(50);
}
```

- Hệ thống hiển thị kết quả lên màn hình LCD 16x2 thông qua giao thức I2C. Điểm đặc biệt của thiết kế là không dùng IP Core I2C của Quartus, mà dùng kỹ thuật Bit-banging bằng phần mềm:
- Điều khiển phân cứng: CPU trực tiếp ghi mức logic (0 hoặc 1) ra hai chân ngoại vi SCL (xung nhịp) và SDA (dữ liệu) thông qua lõi PIO tiêu chuẩn.
- Giả lập giao thức: Các hàm I2C_Start(), I2C_SendByte() sử dụng các lệnh delay_us() xen kẽ với việc lật trạng thái chân tín hiệu để tạo ra dạng sóng chuẩn của I2C.
- Truyền dữ liệu: Hàm LCD_Write_Nibble() chịu trách nhiệm cắt gói dữ liệu 8-bit thành 2 khối 4-bit (Nibble), kết hợp với các cờ điều khiển đèn nền (BL), cờ chốt (EN) và gửi sang chip PCF8574 để hiển thị ký tự.

2.5. Hàm main ()



- Hàm main() là điểm khởi chạy của toàn bộ phần mềm điều khiển trên CPU Nios II. Thay vì chạy từ trên xuống dưới rồi kết thúc, hàm này được thiết kế theo mô hình Máy trạng thái vòng lặp (Polling State Machine).

Lưu đồ hoạt động được chia thành 4 giai đoạn cốt lõi:

Giai đoạn 1: Khởi tạo Hệ thống (Initialization)

- Ngay khi cấp nguồn, CPU thực thi hàm LCD_Init() để cấu hình chuẩn giao tiếp 4-bit và xóa trắng màn hình.
- Các hàm API của thư viện HAL (alt_avalon_sgdma_open) được gọi để liên kết con trỏ phần mềm (sgdma_tx_dev, sgdma_rx_dev) vào đúng địa chỉ vật lý của bộ điều khiển DMA trên kiến trúc FPGA.

Giai đoạn 2: Thiết lập Giao diện Chờ (Idle State)

- Nhấn `start_point`: đóng vai trò là điểm neo (Anchor) của hệ thống.
- Ở trạng thái chờ, toàn bộ dải LED đỏ được tắt (`0x00000`). Màn hình LCD in ra thông báo chào mừng và hướng dẫn sử dụng phím nhấn: *"K0:RST | K1:START"*.

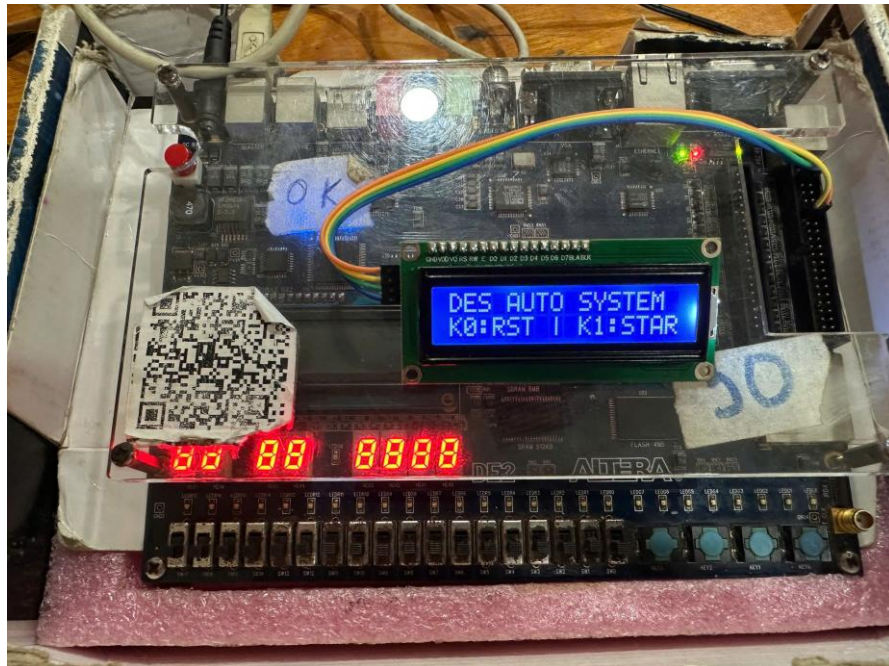
Giai đoạn 3: Giám sát Sự kiện (Event Polling)

- CPU bước vào một vòng lặp vô tận `while(1)`. Tại đây, nó liên tục đọc thanh ghi của khối PIO KEY (`KEY_BASE`) để kiểm tra trạng thái mức logic của các chân nối với nút nhấn cơ học.

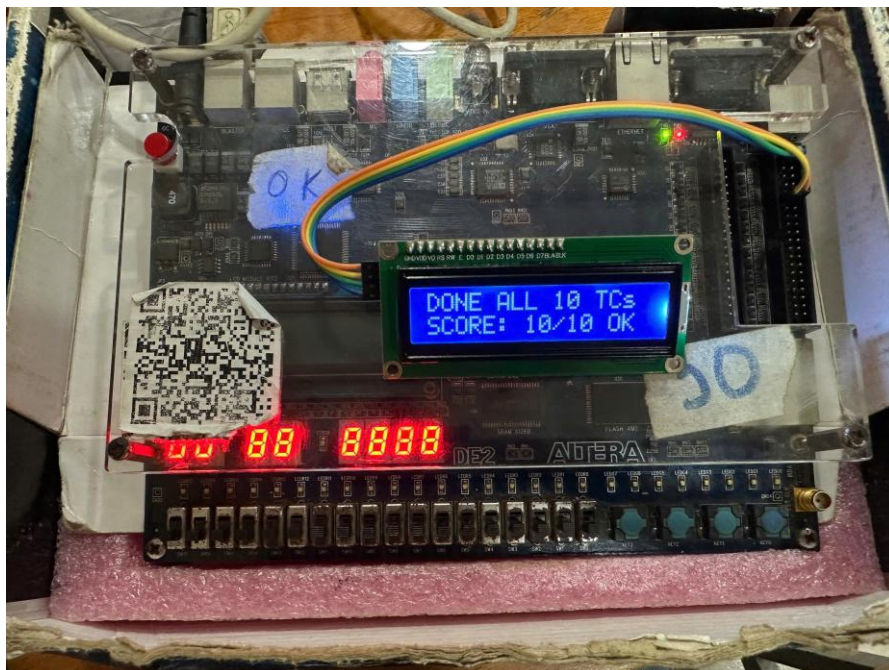
Giai đoạn 4: Lọc Nhiễu và Điều hướng (Debouncing & Routing)

- Khi phát hiện một trong hai phím bị kéo xuống mức logic 0, thuật toán chống dôi phím (Delay 25ms + Vòng lặp khóa chờ nhả phím) được kích hoạt. Sau khi xác nhận sự kiện hợp lệ, hệ thống rẽ nhánh:
- Nếu nhấn `KEY[1]` (Start): Chuyển sang trạng thái bận (Busy). Hệ thống bật sáng toàn bộ 18 LED đỏ (`0x3FFFF`), gọi hàm `Run_All_10_Testcases_Automatically()` để kích hoạt chu trình bơm dữ liệu qua lõi DES. Khi hàm chạy xong, LED tự động tắt.
- Nếu nhấn `KEY[0]` (Reset): Kích hoạt ngắt mềm (Soft-Reset). Lệnh điều khiển hướng dòng nhảy thẳng về nhãn `goto start_point`, tái lập lại trạng thái Idle ban đầu.

III. Hiện thực hệ thống trên DE2 KIT



Hình chờ tín hiệu



Hình đã chạy xong 10 testcase

Link video demo:

<https://drive.google.com/drive/folders/1xAzxcYyeMe0UyRG8O-Da3U8yfmoEofrT?usp=sharing>

IV. Đánh giá

1. Phân công công việc

Tên _ MSSV	Công việc	Đóng góp
Võ Thái Duy _ 23520398	Code v core_Des Code v Des_Avalon_Stream Code c software	50 %
Nguyễn Chí Thanh _ 23521448	Code v core_Des Làm Qsys Code c software	50 %

2. Hướng phát triển trong tương lai

- Sử dụng kiến trúc ngắt (Interrupt-driven) thay vì dùng cơ chế hỏi vòng (Polling) gây lãng phí chu kỳ máy.
- Tính ứng dụng của thuật toán DES: Xét về mặt chuẩn mực an toàn thông tin hiện đại, chiều dài khóa 56-bit của DES đã trở nên lỗi thời và dễ bị phá vỡ bởi các cuộc tấn công vét cạn (Brute-force). Có thể phát triển lên 3DES

V. TÀI LIỆU THAM KHẢO

- [1] Intel Corporation, *Nios II Software Developer's Handbook*, Hướng dẫn phát triển phần mềm, quản lý bộ nhớ Cache và lập trình HAL cho vi xử lý Nios II.
- [2] Intel Corporation, *Avalon® Interface Specifications*, Tài liệu đặc tả kỹ thuật kiến trúc Bus Avalon, bao gồm chuẩn giao tiếp Memory-Mapped (Avalon-MM) và Streaming (Avalon-ST).
- [3] Intel Corporation, *Embedded Peripherals IP User Guide*, Hướng dẫn sử dụng và cấu hình các lõi IP ngoại vi trên Platform Designer (Qsys), bao gồm lõi Parallel I/O (PIO) và lõi Scatter-Gather DMA (SGDMA).
- [4] National Institute of Standards and Technology (NIST), *Data Encryption Standard (DES)*, Federal Information Processing Standards Publication (FIPS PUB) 46-3, 1999. (Tài liệu tiêu chuẩn gốc về thuật toán mã hóa DES).
- [5] Pong P. Chu, *Embedded SoPC Design with Nios II Processor and Verilog Examples*, John Wiley & Sons, 2012. (Sách chuyên khảo về thiết kế hệ thống nhúng SoC trên nền tảng FPGA của Altera).
- [6] NXP Semiconductors, *I2C-bus specification and user manual*, UM10204, Rev. 6.0, 2014. (Tài liệu đặc tả chuẩn giao thức I2C, cơ sở để xây dựng thuật toán Bit-banging).
- [7] Texas Instruments, *PCF8574 Remote 8-Bit I/O Expander for I2C Bus*, Datasheet, 2015. (Tài liệu tra cứu cấu trúc thanh ghi và giản đồ thời gian của IC mở rộng giao tiếp I2C cho màn hình LCD).